

```
!pip install tensorflow tensorflow-hub tensorflow-datasets
```

Show hidden output

```
import tensorflow as tf
import tensorflow_hub as hub
import tensorflow_datasets as tfds

from sklearn import metrics
from mlxtend.plotting import plot_confusion_matrix

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import warnings
warnings.filterwarnings('ignore')
```

```
train_data, validation_data, test_data = tfds.load(
    name='imdb_reviews',
    split=('train[:60%]', 'train[60%:]', 'test'),
    as_supervised=True
)
```

```
train_data, validation_data, test_data = tfds.load(
    name='imdb_reviews',
    split=('train[:60%]', 'train[60%:]', 'test'),
    as_supervised=True
)
```

```
train_examples_batch, train_labels_batch = next(iter(train_data.batch(10)))
train_labels_batch
```

```
<tf.Tensor: shape=(10,), dtype=int64, numpy=array([0, 0, 0, 1, 1, 1, 0, 0, 0, 0])>
```

```
import tensorflow as tf
import tensorflow_hub as hub

embedding = "https://tfhub.dev/google/nnlm-en-dim128-with-normalization/2"

# Create custom wrapper
class HubLayer(tf.keras.layers.Layer):
    def __init__(self, url):
        super().__init__()
        self.hub_layer = hub.KerasLayer(url, trainable=True)

    def call(self, inputs):
        return self.hub_layer(inputs)

# Build model
inputs = tf.keras.Input(shape=(), dtype=tf.string)

x = HubLayer(embedding)(inputs) # ☒ wrapped layer
x = tf.keras.layers.Dense(32, activation='relu')(x)
x = tf.keras.layers.Dense(16, activation='relu')(x)
outputs = tf.keras.layers.Dense(1)(x)

model = tf.keras.Model(inputs, outputs)
```

```
model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None)	0
hub_layer (HubLayer)	(None, 128)	0
dense_3 (Dense)	(None, 32)	4,128
dense_4 (Dense)	(None, 16)	528
dense_5 (Dense)	(None, 1)	17

Total params: 4,673 (18.25 KB)

Trainable params: 4,673 (18.25 KB)

Non-trainable params: 0 (0.00 KB)

```
model.compile(
    optimizer='adam',
    loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),
    metrics=['accuracy']
)
```

```
history = model.fit(
    train_data.shuffle(10000).batch(512),
    epochs=5,
    validation_data=validation_data.batch(512),
    verbose=1
)
```

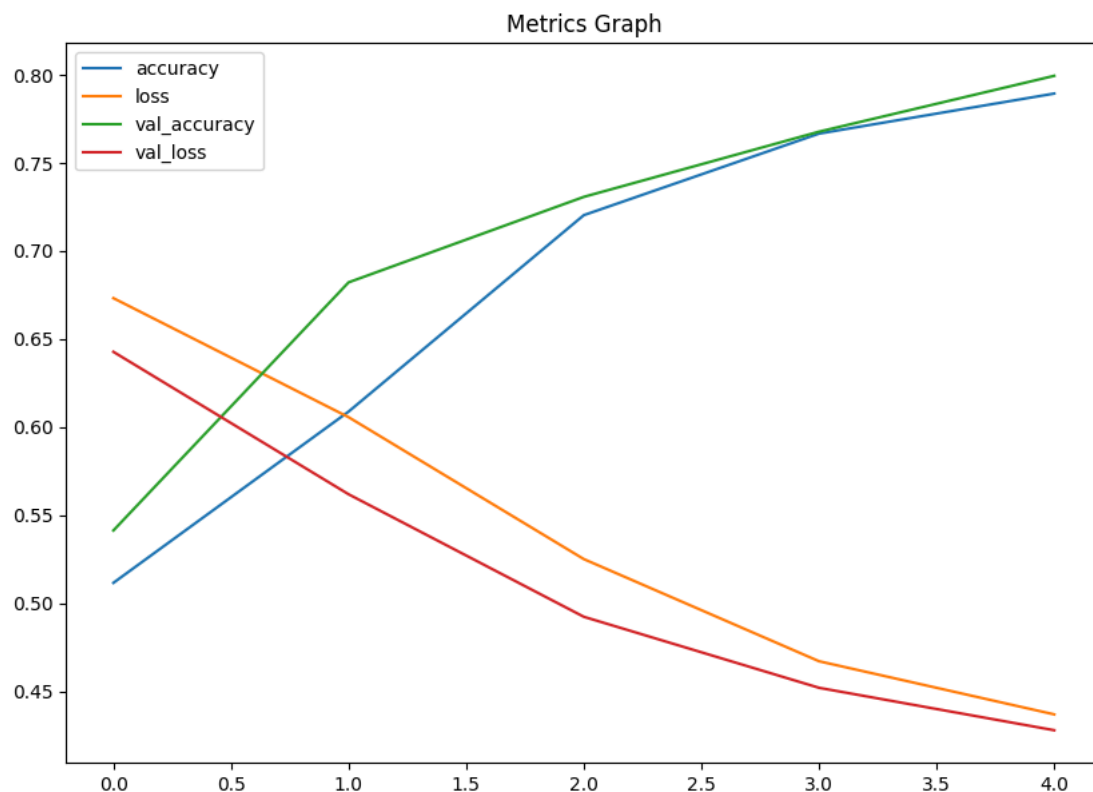
```
Epoch 1/5
30/30 ————— 6s 109ms/step - accuracy: 0.5117 - loss: 0.6732 - val_accuracy: 0.5414 - val_loss: 0.6428
Epoch 2/5
30/30 ————— 4s 143ms/step - accuracy: 0.6089 - loss: 0.6057 - val_accuracy: 0.6822 - val_loss: 0.5619
Epoch 3/5
30/30 ————— 4s 91ms/step - accuracy: 0.7203 - loss: 0.5251 - val_accuracy: 0.7307 - val_loss: 0.4924
Epoch 4/5
30/30 ————— 3s 79ms/step - accuracy: 0.7666 - loss: 0.4672 - val_accuracy: 0.7677 - val_loss: 0.4521
Epoch 5/5
30/30 ————— 3s 81ms/step - accuracy: 0.7893 - loss: 0.4370 - val_accuracy: 0.7994 - val_loss: 0.4280
```

```
results = model.evaluate(test_data.batch(512), verbose=2)
```

```
for name, value in zip(model.metrics_names, results):
    print(f"{name}: {value:.3f}")
```

```
49/49 - 6s - 123ms/step - accuracy: 0.7906 - loss: 0.4371
loss: 0.437
compile_metrics: 0.791
```

```
pd.DataFrame(history.history).plot(figsize=(10,7))
plt.title("Metrics Graph")
plt.show()
```



```
texts = []
true_labels = []

for text, label in test_data:
    texts.append(text.numpy().decode('utf-8')) # convert bytes → string
    true_labels.append(label.numpy())

# Convert labels only (texts will be converted later)
true_labels = np.array(true_labels)
```

```
texts_tensor = tf.constant(texts)

predicted_probs = model.predict(texts_tensor)
predicted_labels = (predicted_probs > 0).astype(int)
```

782/782 ————— 4s 5ms/step

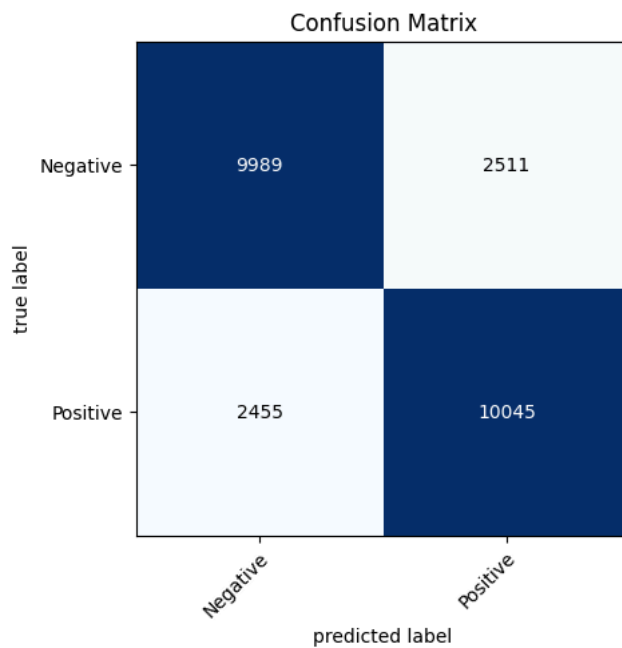
```
report = metrics.classification_report(
    true_labels,
    predicted_labels,
    target_names=['Negative', 'Positive']
)

print(report)
```

	precision	recall	f1-score	support
Negative	0.80	0.80	0.80	12500
Positive	0.80	0.80	0.80	12500
accuracy			0.80	25000
macro avg	0.80	0.80	0.80	25000
weighted avg	0.80	0.80	0.80	25000

```
cm = metrics.confusion_matrix(true_labels, predicted_labels)

plot_confusion_matrix(cm, class_names=['Negative', 'Positive'])
plt.title("Confusion Matrix")
plt.show()
```



Start coding or [generate](#) with AI.